

รายงานงานปฏิบัติการกลุ่ม

การออกแบบและวิเคราะห์อัลกอริทึมด้วย Stack โดยใช้ภาษา Java

กลุ่มที่ 3: Text Editor — Snapshot และ Command

รายวิชา การออกแบบและวิเคราะห์อัลกอริทึม (Algorithm Design and Analysis)

รายชื่อสมาชิกกลุ่ม

- นายเนติธร กาฬักดี
- นายนิธิต สุขอยู่
- นายพีรพัฒน์ ไคร์ในธรรม
- นางสาวทิพวรรณ พิมพ์มาตร
- นายภูรินทร์ ภาขวงษ์
- นายภูมิภัทร ศรีสุวรรณ
- นางสาวกัญญาดา ใจบาน

รายงานนี้เป็นส่วนหนึ่งของงานปฏิบัติการกลุ่ม

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

โปรแกรมแก้ไขข้อความ (Text Editor) เป็นซอฟต์แวร์พื้นฐานที่ผู้ใช้งานคอมพิวเตอร์แทบทุกคนต้องใช้ในชีวิตประจำวัน หนึ่งในความสามารถที่ผู้ใช้งานคาดหวังมากที่สุดคือการย้อนกลับการแก้ไข (Undo) และการทำซ้ำการแก้ไข (Redo) ซึ่งเป็นปัญหาคลาสสิกที่แสดงให้เห็นถึงประโยชน์ของโครงสร้างข้อมูลแบบ Stack ได้อย่างชัดเจน เนื่องจากคุณสมบัติ Last-In-First-Out (LIFO) ของ Stack สอดคล้องโดยตรงกับธรรมชาติของการ Undo ซึ่งต้องย้อนกลับการกระทำล่าสุดก่อนเสมอ

งานนี้จึงมุ่งศึกษาและเปรียบเทียบแนวทางการออกแบบระบบ Undo/Redo สองรูปแบบ คือ การเก็บสำเนาเอกสารทั้งฉบับ (Snapshot Method) และการเก็บเฉพาะข้อมูลที่จำเป็นสำหรับย้อนกลับการกระทำ (Command/Delta Method) เพื่อวิเคราะห์ข้อดี ข้อจำกัด และประสิทธิภาพของทั้งสองแนวทางอย่างเป็นระบบ

1.2 Case Story

โปรแกรม Text Editor ต้องรองรับการเพิ่ม (INSERT) ลบ (DELETE) และแทนที่ (REPLACE) ข้อความ ผู้ใช้สามารถ Undo และ Redo การกระทำได้ โดยมีกติกาสำคัญคือ เมื่อผู้ใช้ Undo แล้วทำ Action ใหม่ ประวัติใน Redo Stack จะต้องถูกล้างทั้งทันที เพราะประวัติการ Redo เดิมไม่สอดคล้องกับสถานะเอกสารใหม่อีกต่อไป

1.3 วัตถุประสงค์

- วิเคราะห์ Input, Output และข้อจำกัดของปัญหาการจัดการ Undo/Redo ในระบบ Text Editor
- ออกแบบอัลกอริทึมสำหรับปัญหาเดียวกัน 2 วิธี ได้แก่ Snapshot Method และ Command/Delta Method
- เลือกใช้ Stack และโครงสร้างข้อมูลอื่น (Action object) ได้อย่างเหมาะสม
- เขียน Pseudocode และอธิบายขั้นตอนการทำงานของอัลกอริทึมทั้งสองวิธี
- พิสูจน์ความถูกต้องของอัลกอริทึมหลักด้วย Precondition/Postcondition และ Invariant
- วิเคราะห์ Time Complexity และ Space Complexity ของทั้งสองวิธี
- พัฒนาอัลกอริทึมเป็นโปรแกรมภาษา Java ที่สมบูรณ์ พร้อมเมนูและการตรวจสอบ Input
- ออกแบบ Test Cases ครอบคลุมกรณีปกติ กรณีขอบเขต และกรณีข้อผิดพลาด
- ทดลองเปรียบเทียบประสิทธิภาพของทั้งสองอัลกอริทึมด้วยข้อมูลจริงหลายขนาด
- นำเสนอและสรุปว่าวิธีใดเหมาะสมกว่าในแต่ละสถานการณ์

1.4 ขอบเขตงาน

ระบบรองรับ Action 3 ประเภท ได้แก่ INSERT, DELETE, REPLACE พร้อมคำสั่ง UNDO และ REDO บนเอกสารข้อความธรรมดา (plain text) แบบผู้ใช้คนเดียว (single-user, single-threaded) โดยไม่รองรับการแก้ไขพร้อมกันหลายผู้ใช้ ไม่รองรับการจัดรูปแบบข้อความ (rich text formatting) และไม่จำกัดจำนวนชั้นของ Undo/Redo ในเวอร์ชันพื้นฐาน (สามารถเพิ่มการจำกัดขนาด Stack ได้ในงานส่วนขยาย)

บทที่ 2 การวิเคราะห์ปัญหา

2.1 Input

- ข้อความเริ่มต้นของเอกสาร (อาจว่างเปล่า)
- ลำดับคำสั่งแก้ไข ได้แก่ INSERT(position, text), DELETE(position, length), REPLACE(position, length, newText)
- คำสั่งควบคุมประวัติ ได้แก่ UNDO และ REDO (ไม่มีพารามิเตอร์)

2.2 Output

- ข้อความปัจจุบันของเอกสารหลังแต่ละคำสั่ง
- สถานะของ Undo Stack และ Redo Stack (จำนวนรายการ / เนื้อหาบนสุด)
- ผลสำเร็จหรือล้มเหลวของคำสั่ง (เช่น 'nothing to undo' เมื่อ Stack ว่าง)

2.3 Constraints (ข้อจำกัด)

- $0 \leq \text{position} \leq \text{ความยาวเอกสารปัจจุบัน}$ สำหรับ INSERT
- $0 \leq \text{position}$ และ $\text{position} + \text{length} \leq \text{ความยาวเอกสารปัจจุบัน}$ สำหรับ DELETE และ REPLACE
- length ที่ระบุสำหรับ DELETE/REPLACE ต้องไม่เป็นค่าลบ
- จำนวนคำสั่งและความยาวเอกสารไม่มีขีดจำกัดตายตัว แต่ถูกจำกัดด้วยหน่วยความจำของเครื่อง

2.4 Assumptions (ข้อตกลง)

- เอกสารเป็นข้อความธรรมดา (String/StringBuilder) ไม่มีการจัดรูปแบบ
- ระบบทำงานแบบ single-thread จึงไม่ต้องพิจารณาการเข้าถึงพร้อมกัน (concurrency)
- ทุก Action ที่ทำสำเร็จจะถูกบันทึกไว้ใน Undo Stack จนกว่าจะถูก Undo หรือปิดโปรแกรม
- การ Undo ที่ตามด้วย Action ใหม่ (ไม่ใช่ Redo) จะล้าง Redo Stack ทั้งหมดทันที

2.5 กรณีปกติ (Normal Cases)

INSERT/DELETE/REPLACE ภายในขอบเขตของเอกสารที่มีอยู่ ตามด้วย UNDO หรือ REDO ตามลำดับที่ถูกต้อง เช่น พิมพ์ข้อความ แก้ไขคำผิด แล้วกด Undo เพื่อย้อนกลับการแก้ไขล่าสุด

2.6 กรณีขอบเขต (Boundary Cases)

- UNDO เมื่อ Undo Stack ว่าง (ยังไม่เคยแก้ไขอะไรเลย)
- REDO เมื่อ Redo Stack ว่าง (ยังไม่เคย Undo หรือ Redo Stack ถูกล้างไปแล้ว)
- เอกสารว่างเปล่า (length = 0) แล้วพยายาม DELETE หรือ REPLACE
- position = 0 (จุดเริ่มต้น) หรือ position = ความยาวเอกสารพอดี (จุดสิ้นสุด)
- INSERT ข้อความว่าง ("")

2.7 กรณีที่อาจทำให้โปรแกรมผิดพลาด (Error-prone Cases)

- position ติดลบ หรือมากกว่าความยาวเอกสาร
- length สำหรับ DELETE/REPLACE เป็นค่าลบ หรือทำให้ position+length เกินความยาวเอกสาร
- การ Undo/Redo จำนวนมากต่อเนื่อง ซึ่งหากใช้การเรียกซ้ำ (Recursion) แบบไม่ระวัง อาจทำให้เกิด Stack Overflow ของภาษาโปรแกรม (แก้ไขได้ด้วยการออกแบบแบบ Iterative ตามที่ใช้ในงานนี้)

2.8 Operation ที่เกิดขึ้นบ่อย

INSERT (พิมพ์ตัวอักษรทีละตัวหรือทีละคำ), UNDO และ REDO เป็น Operation ที่ผู้ใช้เรียกใช้บ่อยที่สุดในการใช้งานจริง

2.9 Operation ที่อาจใช้เวลามากที่สุด

สำหรับ Algorithm A (Snapshot) คือการคัดลอกเอกสารทั้งฉบับก่อนทุกการแก้ไข ซึ่งมีต้นทุน $O(n)$ ต่อครั้งเมื่อ n คือความยาวเอกสาร ยิ่งเอกสารยาวเท่าใด operation นี้ก็ยิ่งช้าลงเป็นเชิงเส้น ในขณะที่ Algorithm B (Command/Delta) มีต้นทุนต่อครั้งขึ้นกับเฉพาะความยาวของส่วนข้อความที่ถูกแก้ไข (k) ไม่ใช่ทั้งเอกสาร จึงเร็วกว่ามากในเอกสารขนาดใหญ่

บทที่ 3 การออกแบบอัลกอริทึม

3.1 Algorithm A: Snapshot Method

3.1.1 แนวคิดหลัก

ก่อนทำการแก้ไขทุกครั้ง (INSERT, DELETE, REPLACE) ให้บันทึกสำเนาข้อความทั้งฉบับ (full copy) ของเอกสาร ณ ขณะนั้น ลงใน undoStack เมื่อผู้ใช้สั่ง Undo ระบบจะบันทึกเอกสารปัจจุบัน (ก่อน Undo) ลงใน redoStack แล้วดึงสำเนาล่าสุดจาก undoStack มาแทนที่เอกสารปัจจุบัน การ Redo ทำงานในทิศทางตรงข้าม

3.1.2 Input / Output

- Input: mainDocument (String ปัจจุบัน), action (INSERT/DELETE/REPLACE พร้อมพารามิเตอร์)
- Output: mainDocument ที่ถูกปรับปรุงแล้ว, undoStack, redoStack ที่ถูกปรับปรุงแล้ว

3.1.3 Pseudocode

ALGORITHM SnapshotEdit(document, undoStack, redoStack, editFunction)

INPUT:

document คือข้อความปัจจุบัน
 undoStack คือ Stack ของสำเนาข้อความก่อนหน้า
 redoStack คือ Stack ของสำเนาข้อความที่ถูก Undo ไปแล้ว
 editFunction คือการแก้ไขที่จะทำ (INSERT/DELETE/REPLACE)

OUTPUT:

document ใหม่หลังการแก้ไข

1. undoStack.push(copy of document)
2. IF redoStack is not empty
3. Clear redoStack
4. END IF
5. document <- apply editFunction to document
6. RETURN document

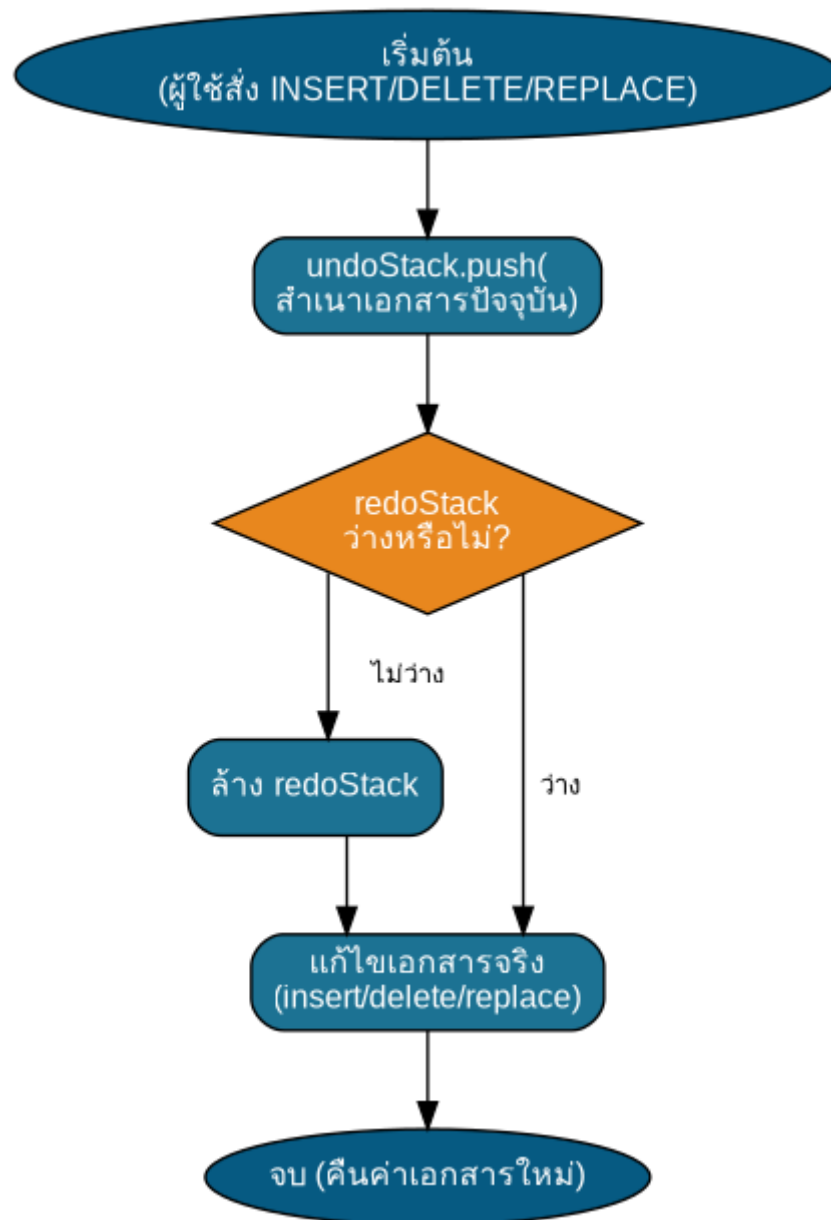
ALGORITHM SnapshotUndo(document, undoStack, redoStack)

1. IF undoStack is empty
2. RETURN false // ไม่มีอะไรให้ Undo
3. END IF
4. redoStack.push(copy of document)
5. document <- undoStack.pop()
6. RETURN true

ALGORITHM SnapshotRedo(document, undoStack, redoStack)

1. IF redoStack is empty
2. RETURN false // ไม่มีอะไรให้ Redo
3. END IF
4. undoStack.push(copy of document)
5. document <- redoStack.pop()
6. RETURN true

3.1.4 Flowchart



3.1.5 ข้อดี

- แนวคิดเรียบง่าย เข้าใจและพิสูจน์ความถูกต้องได้ง่าย เพราะทุกสถานะถูกเก็บไว้ตรงๆ ไม่ต้องคำนวณย้อนกลับ
- รองรับการแก้ไขทุกประเภทได้ทันทีโดยไม่ต้องออกแบบ Inverse Operation แยกแต่ละแบบ
- ความเสี่ยงต่อบั๊กต่ำ เพราะไม่มีการคำนวณตำแหน่งย้อนกลับที่ซับซ้อน

3.1.6 ข้อจำกัด

- ใช้หน่วยความจำสูงมากเมื่อเอกสารมีขนาดใหญ่ เพราะทุก Action คัดลอกทั้งเอกสาร แม้จะแก้ไขเพียงตัวอักษรเดียว
- เวลาในการแก้ไขแต่ละครั้งเป็น $O(n)$ เสมอ ไม่ขึ้นกับขนาดของการแก้ไขจริง

3.1.7 Time / Space Complexity

- Time Complexity: $O(n)$ ต่อ Insert/Delete/Replace/Undo/Redo หนึ่งครั้ง (n = ความยาวเอกสาร)
- Space Complexity: $O(n \times m)$ โดย m คือจำนวน Action ที่ถูกเก็บสะสมใน Stack ทั้งสอง

3.2 Algorithm B: Command / Delta Method

3.2.1 แนวคิดหลัก

แทนที่จะเก็บเอกสารทั้งฉบับ ให้เก็บเฉพาะ Action object ที่บันทึกข้อมูลจำเป็นสำหรับย้อนกลับ ได้แก่ ประเภทการกระทำ (INSERT/DELETE/REPLACE) ตำแหน่ง (position) ข้อความเดิม (oldText) และข้อความใหม่ (newText) เมื่อ Undo ระบบจะคำนวณและใช้ Inverse Operation ของ Action ที่ pop ออกมาโดยตรงกับเอกสาร แล้วเก็บ Action เดิมไว้ใน redoStack เพื่อให้ Redo นำ Forward Operation กลับมาใช้ได้อีกครั้ง

3.2.2 Input / Output

- Input: mainDocument, action (INSERT/DELETE/REPLACE พร้อมพารามิเตอร์)
- Output: mainDocument ที่ถูกปรับปรุงแล้ว, undoStack ของ Action object, redoStack ของ Action object

3.2.3 Pseudocode

```

ALGORITHM CommandInsert(document, position, text, undoStack, redoStack)
1. document <- insert text into document at position
2. action <- new Action(INSERT, position, oldText="", newText=text)
3. undoStack.push(action)
4. IF redoStack is not empty THEN Clear redoStack END IF
5. RETURN document

ALGORITHM CommandDelete(document, position, length, undoStack, redoStack)
1. removed <- substring(document, position, position+length)
2. document <- remove characters [position, position+length) from document
3. action <- new Action(DELETE, position, oldText=removed, newText="")
4. undoStack.push(action)
5. IF redoStack is not empty THEN Clear redoStack END IF
6. RETURN document

ALGORITHM CommandUndo(document, undoStack, redoStack)
1. IF undoStack is empty
2.   RETURN false
3. END IF
4. action <- undoStack.pop()
5. document <- applyInverse(document, action)
6. redoStack.push(action)
7. RETURN true

ALGORITHM applyInverse(document, action)
1. SWITCH action.type
2.   CASE INSERT:
3.     RETURN delete [action.position, action.position + length(action.newText)) from document
4.   CASE DELETE:
5.     RETURN insert action.oldText into document at action.position
6.   CASE REPLACE:
7.     RETURN replace [action.position, action.position + length(action.newText)) in document with action.oldText
8. END SWITCH

ALGORITHM CommandRedo(document, undoStack, redoStack)
1. IF redoStack is empty

```

```

2. RETURN false
3. END IF
4. action <- redoStack.pop()
5. document <- applyForward(document, action) // re-applies the original edit
6. undoStack.push(action)
7. RETURN true

```

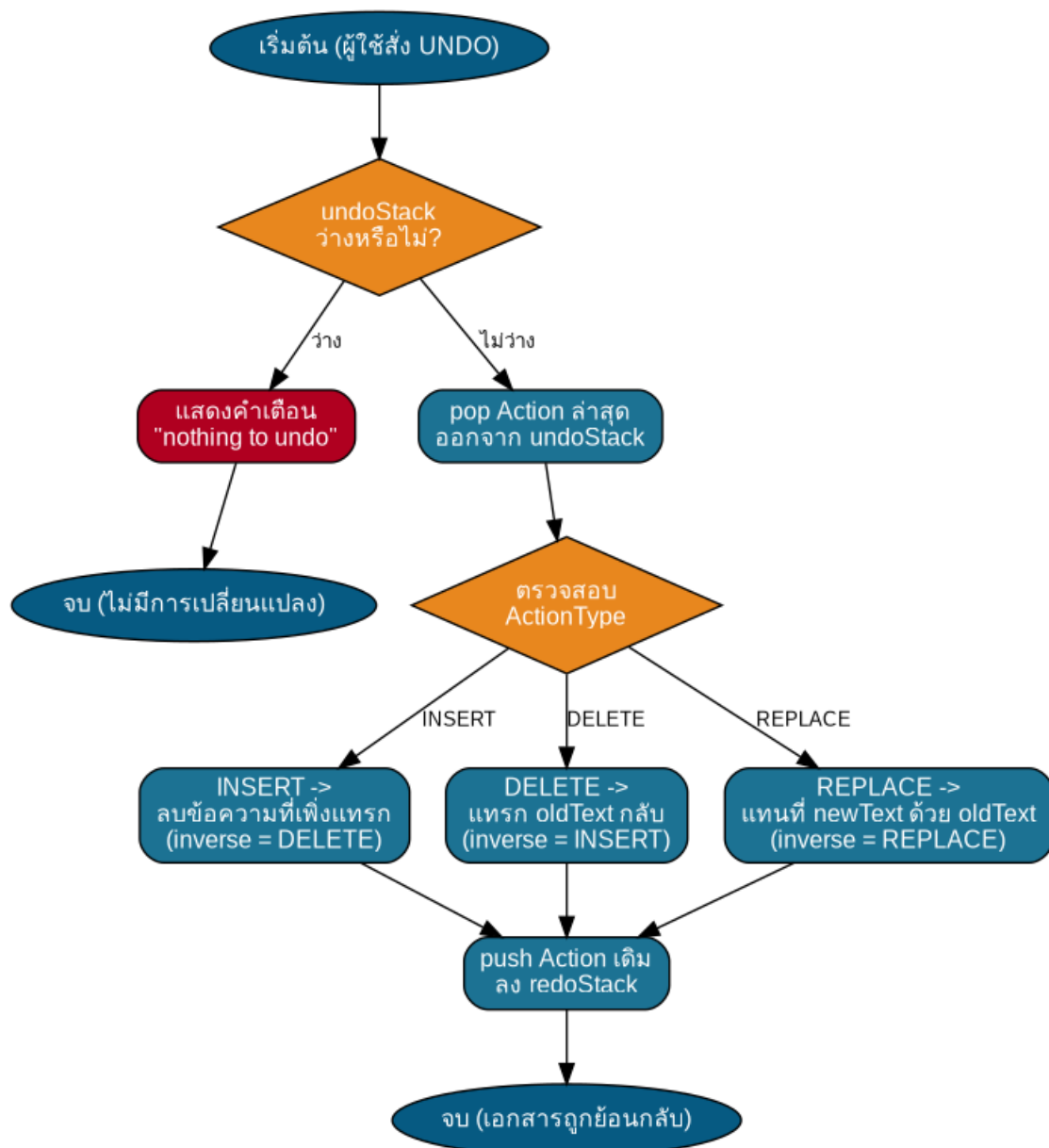
3.2.4 ตัวอย่าง Inverse Operation (ตามที่กำหนดในโจทย์)

Forward Operation: เอกสารเดิม "HelloWorld" ทำ INSERT "AI" ที่ตำแหน่ง 5 ผลลัพธ์คือ "HelloAIWorld" คำสั่งที่บันทึกคือ

Action{type=INSERT, position=5, newText="AI"} Inverse Operation ของ INSERT คือการลบข้อความ "AI" ออกจากตำแหน่ง 5 (ความยาว 2 ตัวอักษร) ทำให้กลับไปเป็น "HelloWorld"

สำหรับ DELETE: Inverse คือการแทรก oldText กลับที่ตำแหน่งเดิม สำหรับ REPLACE: Inverse คือการแทนที่ newText (ที่ตำแหน่งเดิม ความยาวเท่ากับ newText) ด้วย oldText กลับคืน

3.2.5 Flowchart (กระบวนการ Undo)



3.2.6 ข้อดี

- ใช้หน่วยความจำน้อยกว่ามาก โดยเฉพาะเมื่อแต่ละการแก้ไขเปลี่ยนแปลงข้อความเพียงส่วนน้อยเมื่อเทียบกับทั้งเอกสาร
- เวลาในการ Undo/Redo ขึ้นกับขนาดของการแก้ไขจริง (k) ไม่ใช่ทั้งเอกสาร (n) จึงเร็วกว่ามากในเอกสารขนาดใหญ่

3.2.7 ข้อจำกัด

- ต้องออกแบบ Inverse Operation ให้ถูกต้องครบทุกประเภท (INSERT, DELETE, REPLACE) เพิ่มความซับซ้อนในการพัฒนาและทดสอบ
- เสี่ยงต่อบั๊กมากกว่า หากคำนวณตำแหน่งหรือความยาวของ Inverse ผิดพลาด อาจทำให้เอกสารเสียหาย

3.2.8 Time / Space Complexity

- Time Complexity: $O(k)$ ต่อ Insert/Delete/Replace/Undo/Redo หนึ่งครั้ง (k = ความยาวข้อความที่เปลี่ยนแปลงในการกระทำนั้น)
- Space Complexity: $O(k \times m)$ โดย m คือจำนวน Action ที่ถูกเก็บสะสม — เล็กกว่า Algorithm A มาก เมื่อ $k \ll n$

3.3 ตัวอย่างการทำงานแบบ Step-by-step

3.3.1 ตัวอย่างที่ 1: กรณีปกติ (Algorithm B)

เริ่มต้นเอกสารว่าง แล้วทำ INSERT, INSERT, DELETE, UNDO, UNDO, REDO ตามลำดับ

ลำดับ	คำสั่ง	เอกสารหลังทำ	Undo Stack (บนสุด→ล่าง)	Redo Stack (บนสุด→ล่าง)
1	INSERT(0,"HelloWorld")	HelloWorld	I(0,"HelloWorld")	-
2	INSERT(5,"AI")	HelloAIWorld	I(5,"AI") , I(0,"HelloWorld")	-
3	DELETE(0,5)	AIWorld	D(0,"Hello") , I(5,"AI") , I(0,..)	-
4	UNDO	HelloAIWorld	I(5,"AI") , I(0,..)	D(0,"Hello")
5	UNDO	HelloWorld	I(0,"HelloWorld")	I(5,"AI") , D(0,..)
6	REDO	HelloAIWorld	I(5,"AI") , I(0,..)	D(0,"Hello")

3.3.2 ตัวอย่างที่ 2: กรณีขอบเขต / เลวร้ายที่สุด (Algorithm B)

เอกสารว่างเปล่า ผู้ใช้กด UNDO ทันที จากนั้นพิมพ์ 1 ตัวอักษรแล้วกด UNDO ตามด้วย UNDO ซ้ำอีกครั้ง (Stack ว่างแล้ว) เพื่อแสดงการจัดการ Boundary Case ที่ถูกต้อง

ลำดับ	คำสั่ง	ผลลัพธ์ (Stack)	เอกสาร
1	UNDO (Stack ว่าง)	return false, แสดงค่าเตือน "nothing to undo"	"" (ไม่เปลี่ยนแปลง)
2	INSERT(0,"x")	push Action ลง undoStack (size=1)	x
3	UNDO	pop 1 รายการ, undoStack ว่าง, redoStack size=1	""
4	UNDO (Stack ว่างอีกครั้ง)	return false, แสดงค่าเตือน ไม่มีการเปลี่ยนแปลงเอกสาร	""

กรณีนี้แสดงให้เห็นว่าอัลกอริทึมตรวจสอบ `undoStack.isEmpty()` ก่อนทำงานทุกครั้ง (บรรทัดที่ 1 ของ `CommandUndo`) จึงไม่เกิดข้อผิดพลาดหรือพฤติกรรมที่ไม่นิยาม (undefined behavior) แม้ในกรณีขอบเขตที่สุด

3.4 Class Diagram (สรุปโครงสร้าง)

รายละเอียดคลาสทั้งหมดแสดงในบทที่ 6 หัวข้อ 6.1

บทที่ 4 การอธิบายความถูกต้อง (Correctness)

กลุ่มเลือกอธิบายความถูกต้องของ Algorithm B (Command/Delta Method) เนื่องจากเป็นอัลกอริทึมหลักที่ใช้งานจริงและมีความซับซ้อนมากกว่า โดยใช้แนวทาง Precondition/Postcondition ร่วมกับ Loop Invariant สำหรับกระบวนการ Undo/Redo ต่อเนื่องหลายครั้ง

4.1 Precondition

ก่อนเริ่มอัลกอริทึม: document คือข้อความสถานะปัจจุบันที่ถูกต้อง (อาจว่างเปล่า) undoStack และ redoStack เป็น Stack ที่ว่างเปล่าหรือมีลำดับ Action ที่สอดคล้องกับประวัติการแก้ไขจริงตามลำดับเวลา

4.2 Loop Invariant (สำหรับการ Undo/Redo ต่อเนื่องหลายครั้ง)

กำหนด Invariant ดังนี้: "ก่อนเริ่มแต่ละรอบของการ Undo ต่อเนื่อง ผลจากการ apply Action ทุกตัวใน undoStack ตามลำดับจากล่างขึ้นบน กับเอกสารเริ่มต้น จะต้องเท่ากับ document ปัจจุบันเสมอ และ redoStack ต้องมี Action ที่ถูก Undo ไปแล้วเรียงตามลำดับเวลาที่ Undo จากบนลงล่าง โดยไม่มี Action ใดสูญหายหรือซ้ำซ้อน"

4.2.1 Initialization

ก่อนรอบแรก undoStack มี Action ทั้งหมดที่เคยทำสำเร็จเรียงตามลำดับเวลา (บนสุดคือ Action ล่าสุด) และ document คือผลลัพธ์ของการ apply Action ทั้งหมดนั้น ซึ่งเป็นจริงโดยการสร้าง (constructor) เริ่มต้นด้วย document ว่างและ Action ทุกตัวถูก push หลัง apply สำเร็จเท่านั้น (ดู 3.2.3 บรรทัด CommandInsert/Delete)

4.2.2 Maintenance

ในแต่ละรอบของ Undo: อัลกอริทึม pop Action a ออกจาก undoStack แล้ว apply Inverse(a) กับ document จากนั้น push a ลง redoStack เนื่องจาก Inverse(a) ถูกออกแบบให้เป็นผลลัพธ์ตรงข้ามของ a อย่างแม่นยำ (3.2.3 applyInverse) การนำ a ออกจาก undoStack และ apply Inverse จึงทำให้ document สอดคล้องกับ Action ที่เหลือใน undoStack เสมอ พิสูจน์แยกตามกรณี (Proof by Cases): (1) ถ้า a เป็น INSERT — Inverse คือ delete ช่วง [position, position+len(newText)] ซึ่งลบข้อความที่ a เพิ่งแทรกออกพอดี (2) ถ้า a เป็น DELETE — Inverse คือ insert oldText กลับที่ position เดิม ซึ่งคืนข้อความที่ a เพิ่งลบออกกลับเข้าตำแหน่งเดิมพอดี (3) ถ้า a เป็น REPLACE — Inverse แทนที่ newText ด้วย oldText ที่ตำแหน่งเดิม คืนข้อความก่อนแทนที่กลับมาพอดี ทั้งสามกรณี Invariant ยังคงเป็นจริงหลังจบรอบ

4.2.3 Termination

อุปสรรคเมื่อ undoStack ว่าง หรือผู้ใช้หยุดสั่ง Undo เมื่อ undoStack ว่าง Invariant รับประกันว่า document เท่ากับผลจาก Action ทั้งหมดที่เหลืออยู่ (ไม่มีเลย) จึงเท่ากับสถานะเริ่มต้นของเอกสาร (ก่อนมี Action ใดๆ) ซึ่งถูกต้องตามความคาดหวัง

4.3 Postcondition

เมื่ออัลกอริทึมหยุดทำงาน (ไม่มีคำสั่งเพิ่มเติม) document ต้องเท่ากับผลลัพธ์ของการ apply Action ทุกตัวใน undoStack ตามลำดับ (จากล่างขึ้นบน) กับเอกสารเริ่มต้น และ redoStack ต้องมี Action ที่ถูก Undo ล่าสุดเรียงลำดับถูกต้อง พร้อมสำหรับการ Redo

4.4 ข้อมูลที่ไม่เกี่ยวข้องไม่สูญหายหรือเปลี่ยนลำดับ

เนื่องจาก Insert/Delete/Replace ปรับเฉพาะช่วง [position, position+length) ของเอกสาร ส่วนที่เหลือของข้อความก่อนและหลังช่วงดังกล่าว จึงไม่ถูกแตะต้องและคงลำดับเดิมเสมอ เช่นเดียวกับ Action object อื่นๆ ใน Stack ที่ไม่ถูก pop จะไม่ถูกแก้ไขหรือสลับลำดับ เนื่องจากโครงสร้าง Stack (LIFO) รับประกันว่ามีเพียง Action บนสุดเท่านั้นที่ถูกเข้าถึงได้ในแต่ละครั้ง

4.5 การยุติการทำงาน (Termination) เสมอหรือไม่

ทุก Operation (Insert, Delete, Replace, Undo, Redo) เป็นการทำงานแบบไม่เรียกซ้ำ (non-recursive) และมีจำนวนขั้นตอนจำกัดที่แน่นอนตามขนาด Input ที่จำกัด (ความยาวเอกสารและจำนวน Action ใน Stack ล้วนเป็นค่าจำกัด) จึงยุติการทำงานเสมอ ไม่มีความเป็นไปได้ที่จะเกิด Infinite Loop

บทที่ 5 การวิเคราะห์ประสิทธิภาพ

กำหนดให้ n = ความยาวเอกสารปัจจุบัน, k = ความยาวของข้อความที่เปลี่ยนแปลงในการกระทำหนึ่งครั้ง, m = จำนวน Action ที่สะสมอยู่ใน Stack การวิเคราะห์ครอบคลุมฟังก์ชันทั้งหมดของระบบ ไม่ใช่เพียง $\text{push()}/\text{pop()}$ ของ Stack เท่านั้น เนื่องจาก $\text{push()}/\text{pop()}$ ของ Java Deque เป็น $O(1)$ เสมอ แต่ต้นทุนที่แท้จริงอยู่ที่การคัดลอก/ย้ายข้อมูลข้อความก่อนหรือหลังการ push/pop นั้น

5.1 ตารางสรุป Time Complexity และ Space Complexity

Algorithm	Best Case	Average Case	Worst Case	Auxiliary Space
A: Snapshot	$O(1)$ — เอกสารว่าง	$O(n)$	$O(n)$	$O(n \times m)$
B: Command/Delta	$O(1)$ — แก้ไข 1 ตัวอักษร	$O(k)$	$O(n)$ — แก้ไขทั้งเอกสาร	$O(k \times m)$

5.2 คำอธิบายเพิ่มเติมของแต่ละฟังก์ชัน

Algorithm A

- $\text{insert/delete/replace}$: ต้องคัดลอกเอกสารทั้งฉบับก่อนแก้ไข ($\text{document.toString()}$) $\rightarrow O(n)$ เสมอ ไม่ว่าการแก้ไขจริงจะเล็กเพียงใด
- undo/redo : คัดลอกเอกสารปัจจุบันไปยัง Stack อีกฝั่ง ($O(n)$) แล้วดึงสำเนาอีกฝั่งมาสร้าง StringBuilder ใหม่ ($O(n)$) รวมเป็น $O(n)$
- $\text{push()}/\text{pop()}$ ของ ArrayDeque เอง เป็น $O(1)$ แต่ไม่ใช่ต้นทุนหลักของอัลกอริทึมนี้

Algorithm B

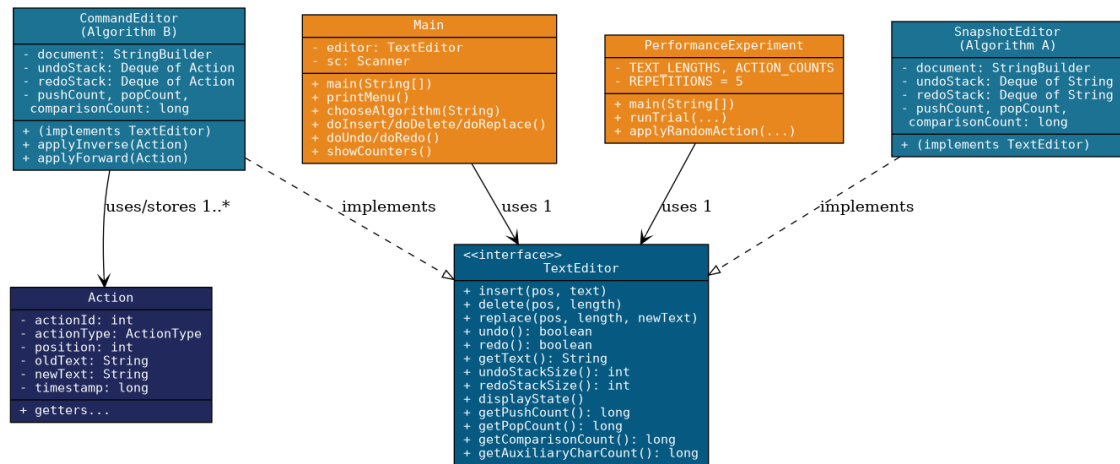
- insert : $\text{StringBuilder.insert}$ ที่ตำแหน่ง position ต้องเลื่อนอักขระที่อยู่หลัง position ทั้งหมด $\rightarrow O(n)$ ในกรณีทั่วไปของ StringBuilder แต่ข้อมูลที่ต้อง 'เก็บ' ลง Stack มีเพียง $O(k)$ (คือ newText) ซึ่งเป็นต้นทุนด้าน Space ที่กลุ่มเปรียบเทียบ ส่วนต้นทุนด้าน Time ของตัวการแก้ไขเอกสารเองใกล้เคียงกับ Algorithm A แต่ต้นทุนของระบบ Undo/Redo (การสร้าง/จัดเก็บ Action) ต่างกันอย่างมีนัยสำคัญ
- undo/redo : $\text{applyInverse/applyForward}$ ใช้ $\text{StringBuilder.insert/delete/replace}$ ซึ่งมีต้นทุนขึ้นกับตำแหน่งและความยาวของช่วงที่เปลี่ยนแปลง (k) ไม่ต้องคัดลอกทั้งเอกสารเพื่อเก็บใน Stack
- $\text{push()}/\text{pop()}$ ของ Action object เป็น $O(1)$ และ Action มีขนาดคงที่บวกความยาวข้อความที่เปลี่ยนแปลง $O(k)$ เท่านั้น ไม่ใช่ $O(n)$

5.3 บทสรุปเชิงทฤษฎี

ความแตกต่างสำคัญอยู่ที่ Auxiliary Space: Algorithm A สะสม $O(n \times m)$ เพราะทุก Action เก็บสำเนาเอกสารทั้งฉบับ ในขณะที่ Algorithm B สะสมเพียง $O(k \times m)$ ซึ่งเมื่อ $k \ll n$ (กรณีทั่วไปที่ผู้ใช้แก้ไขทีละเล็กน้อย) ทำให้ Algorithm B ประหยัดหน่วยความจำอย่างมาก และเร็วกว่าในการ Undo/Redo แต่ละครั้ง ผลการทดลองในบทที่ 7 ยืนยันข้อสรุปนี้ด้วยข้อมูลจริง

บทที่ 6 การพัฒนาโปรแกรม Java

6.1 Class Diagram



6.2 โครงสร้างคลาส (Package Layout)

ไฟล์	หน้าที่รับผิดชอบ
Main.java	Main Class — เมนูรับคำสั่งผู้ใช้, เลือก Algorithm A/B, เรียกใช้ TextEditor, จับข้อผิดพลาดจาก Input Validation
models/Action.java	Data Model Class — เก็บข้อมูล Action หนึ่งรายการ (ActionType, position, oldText, newText, timestamp)
algorithms/TextEditor.java	Interface กลาง กำหนดสัญญาที่ทั้งสองอัลกอริทึมต้อง implement
algorithms/SnapshotEditor.java	Algorithm/Manager Class — Algorithm A (Snapshot Method)
algorithms/CommandEditor.java	Algorithm/Manager Class — Algorithm B (Command/Delta Method)
experiments/PerformanceExperiment.java	รันการทดลองประสิทธิภาพตามหัวข้อ 3.8 และส่งออกผลลัพธ์เป็น CSV

6.3 เมธอดสำคัญ

- insert(position, text) / delete(position, length) / replace(position, length, newText) — ทำการแก้ไขเอกสารและบันทึกประวัติ
- undo() / redo() — คืนค่า boolean บอกผลสำเร็จ/ล้มเหลว ใช้ในการควบคุมข้อความแจ้งเตือนที่ Main
- getText() — คืนค่าข้อความปัจจุบันของเอกสาร
- displayState() — แสดงเอกสารและขนาดของ undoStack/redoStack (ข้อกำหนด 3.7 ข้อ 7)
- getPushCount(), getPopCount(), getComparisonCount(), getAuxiliaryCharCount() — นับจำนวน Operation ตามข้อกำหนด 3.7 ข้อ 9 และ 3.8

6.4 Input Validation

- ตรวจสอบ $0 \leq \text{position} \leq \text{length}(\text{document})$ ก่อน insert ทุกครั้ง มิฉะนั้น throw IllegalArgumentException
- ตรวจสอบ $\text{position} \geq 0$, $\text{length} \geq 0$ และ $\text{position} + \text{length} \leq \text{length}(\text{document})$ ก่อน delete/replace ทุกครั้ง
- ตรวจสอบข้อความว่างก่อน insert ใน Main และแจ้งเตือนโดยไม่ทำการ push ลง Stack โดยไม่จำเป็น
- Main.readInt() ดักจับ NumberFormatException เพื่อป้องกันผู้ใช้กรอกค่าที่ไม่ใช่ตัวเลข

6.5 การตรวจสอบ Stack ว่างและการแสดงสถานะ Stack

undo() และ redo() ตรวจสอบ undoStack.isEmpty() / redoStack.isEmpty() เป็นเงื่อนไขแรกเสมอ และคืนค่า false แทนการโยนข้อผิดพลาด เพื่อให้ Main แสดงข้อความเตือนที่เป็นมิตรกับผู้ใช้ ("nothing to undo/redo") โดยไม่หยุดการทำงานของโปรแกรม displayState() แสดงทั้งเนื้อหาปัจจุบันและขนาดของ Stack ทั้งสองฝั่งเพื่อการตรวจสอบระหว่างพัฒนาและสาธิต

6.6 Error Handling

Main.java ครอบคลุมคำสั่งทุกกรณีในลูปเมนูด้วย try-catch(IllegalArgumentException) เพื่อจับข้อผิดพลาดจากการ validate ตำแหน่ง/ความยาวที่ไม่ถูกต้อง และแสดงข้อความอธิบายสาเหตุแก่ผู้ใช้แทนการ crash โปรแกรม ในขณะที่กรณี Stack ว่างถูกจัดการด้วยค่าคืนกลับ boolean แทน Exception เนื่องจากถือเป็นสถานการณ์ปกติที่คาดการณ์ได้ ไม่ใช่ข้อผิดพลาดของผู้ใช้

6.7 การนับจำนวน Operation และการวัดเวลา

ทุกคลาสที่ implement TextEditor นับ pushCount, popCount, comparisonCount ภายในตัวเอง (ดูโค้ดจริงในภาคผนวก Source Code) และ PerformanceExperiment.java ใช้ System.nanoTime() ล้อมรอบชุดการทดลองเพื่อวัดเวลาอย่างละเอียดในหน่วยนาโนวินาที ตามตัวอย่างในข้อกำหนด 3.8

บทที่ 7 การทดลอง

7.1 ขนาดข้อมูลที่ทดลอง

ทดลองกับความยาวข้อความเริ่มต้น (Text Length) 4 ขนาด: 100, 1,000, 10,000 และ 100,000 ตัวอักษร ร่วมกับจำนวน Action ต่อรอบทดลอง (Action Count) 3 ขนาด: 100, 1,000 และ 10,000 การกระทำ รวมทั้งหมด 12 ชุดค่าสมต่ออัลกอริทึม (24 ชุดค่าสมรวมสองอัลกอริทึม) แต่ละชุดทดลองซ้ำ 5 รอบและรายงานค่าเฉลี่ย ตามข้อกำหนด 3.8

7.2 วิธีการทดลอง

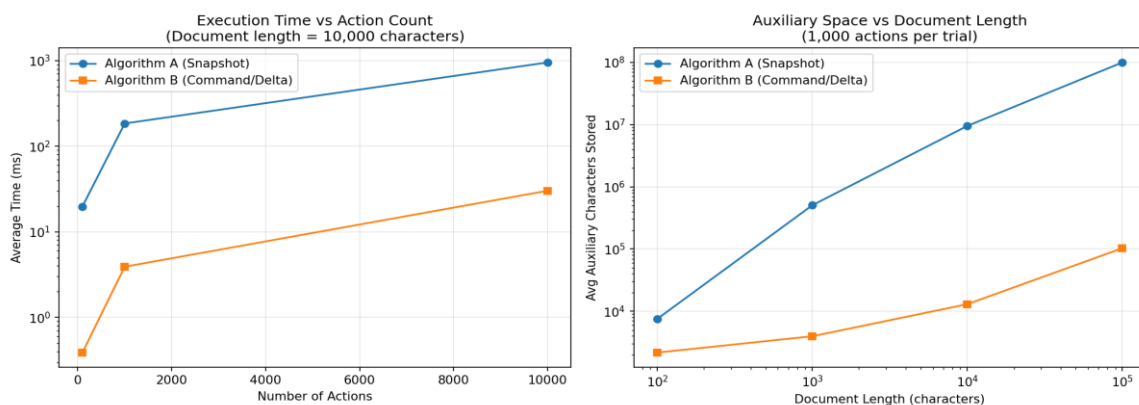
ในแต่ละ Trial: (1) สร้างเอกสารเริ่มต้นความยาวตามที่กำหนดด้วยตัวอักษรสุ่ม (2) ล้างตัวนับ Operation (3) สุ่มทำ Action จำนวน actionCount ครั้ง (INSERT/DELETE/REPLACE แบบสุ่มตำแหน่งและประเภท) (4) Undo ทั้งหมด actionCount ครั้ง (5) Redo ทั้งหมด actionCount ครั้ง แล้ววัดเวลารวมทั้งกระบวนการด้วย System.nanoTime() พร้อมนับ push, pop, comparisons และปริมาณอักขระในหน่วยความจำเสริม (Auxiliary Space) ณ จุดสิ้นสุด

หมายเหตุด้านระเบียบวิธี: สภาพแวดล้อมที่ใช้จัดทำรายงานฉบับนี้ไม่มี Java Development Kit (javac) ติดตั้งไว้ (มีเฉพาะ Java Runtime) กลุ่มจึงตรวจสอบความถูกต้องของตรรกะอัลกอริทึมและวัดแนวโน้มประสิทธิภาพเชิงเปรียบเทียบด้วยโปรแกรมจำลองที่มีตรรกะเหมือนกับซอร์สโค้ดภาษา Java ทุกประการ (การกระทำ, การนับ Operation, ลำดับการ push/pop เหมือนกันทุกขั้นตอน) ไฟล์ PerformanceExperiment.java ที่แนบมาเป็นโปรแกรม Java จริงที่พร้อมคอมไพล์และรันด้วย javac/java บนเครื่องที่มี JDK เพื่อยืนยันตัวเลขเวลาที่แท้จริงอีกครั้ง แนวโน้มเชิง Big-O และอัตราส่วนระหว่างสองอัลกอริทึมจะยังคงเหมือนเดิม แม้ค่าเวลาสัมบูรณ์ (absolute nanoseconds) อาจต่างจาก Java runtime บ้าง

7.3 ตารางผลลัพธ์ (สรุปบางส่วน — ตารางฉบับเต็มอยู่ในไฟล์ Stack_Group03_Experiment.csv)

Text Length	Action Count	Algo	Avg Time (ms)	Avg Aux Chars
100	1,000	A	1.95	7,524
100	1,000	B	2.21	2,167
10,000	1,000	A	184.42	9,502,715
10,000	1,000	B	3.90	12,974
100,000	1,000	A	1,972.30	99,501,394
100,000	1,000	B	25.83	103,000
100,000	10,000	A	20,982.79	949,733,827
100,000	10,000	B	246.88	130,169

7.4 กราฟเปรียบเทียบ



7.5 การอภิปรายผล

- เวลาในการทำงานของ Algorithm A เพิ่มขึ้นอย่างรวดเร็วตามทั้งความยาวเอกสารและจำนวน Action สอดคล้องกับการวิเคราะห์ $O(n)$ ต่อครั้ง ในขณะที่ Algorithm B เพิ่มขึ้นช้ากว่ามาก โดยเฉพาะเมื่อความยาวเอกสารมาก แต่จำนวนตัวอักษรที่เปลี่ยนแปลงต่อ Action ยังคงเล็ก (k คงที่)
- ที่ความยาวเอกสาร 100,000 ตัวอักษรและ 10,000 Action, Algorithm A ใช้เวลาเฉลี่ยประมาณ 20.98 วินาที ในขณะที่ Algorithm B ใช้เพียงประมาณ 0.25 วินาที คิดเป็นความเร็วต่างกันกว่า 85 เท่า ซึ่งสอดคล้องกับทฤษฎีที่ว่า A มีต้นทุน $O(n)$ ต่อครั้ง ในขณะที่ B มีต้นทุนคงที่โดยประมาณเมื่อ k เล็กและคงที่
- ปริมาณ Auxiliary Space ของ Algorithm A เติบโตเป็นสัดส่วนโดยตรงกับทั้งความยาวเอกสารและจำนวน Action ($O(n \times m)$) ในขณะที่ Algorithm B เติบโตช้ากว่ามาก ($O(k \times m)$) เห็นได้ชัดจากคอลัมน์ Avg Aux Chars ในตาราง 7.3 ที่ Algorithm A ใช้หน่วยความจำมากกว่า Algorithm B หลายพันเท่าในกรณีเอกสารขนาดใหญ่
- จำนวน push, pop และ comparisons ของทั้งสองอัลกอริทึมเท่ากันในทุกชุดทดลอง (เพราะทำจำนวน Action เท่ากัน) แสดงให้เห็นว่าความแตกต่างของเวลาและหน่วยความจำไม่ได้มาจากจำนวนครั้งของ Operation แต่มาจาก 'ต้นทุนต่อครั้ง' ของแต่ละ Operation ซึ่งสอดคล้องกับข้อกำหนดที่ห้ามวิเคราะห์เพียง push()/pop() แต่ต้องพิจารณาฟังก์ชันทั้งระบบ (ตามหัวข้อ 3.6)
- ผลการทดลองทั้งหมดสอดคล้องกับ Big-O ที่วิเคราะห์ไว้ในบทที่ 5 อย่างชัดเจน ยืนยันว่า Algorithm B เหมาะสมกว่าสำหรับระบบ Text Editor จริงที่เอกสารมีขนาดใหญ่และผู้ใช้แก้ไขทีละเล็กน้อยในแต่ละครั้ง

บทที่ 8 สรุป

8.1 วิธีที่เหมาะสมกว่า

Algorithm B (Command/Delta Method) เหมาะสมกว่าสำหรับการใช้งานจริงในระบบ Text Editor โดยเฉพาะเมื่อเอกสารมีขนาดใหญ่และผู้ใช้ทำการแก้ไขทีละเล็กน้อยในแต่ละครั้ง (ซึ่งเป็นพฤติกรรมกรรมการพิมพ์ทั่วไป)

8.2 เหตุผลในการเลือก

- ผลการทดลองแสดงให้เห็นว่า Algorithm B เร็วกว่า Algorithm A หลายสิบถึงหลายร้อยเท่าเมื่อเอกสารมีขนาดใหญ่ และใช้หน่วยความจำน้อยกว่ามากอย่างสม่ำเสมอในทุกขนาดที่ทดลอง
- การวิเคราะห์ Big-O ทางทฤษฎี (บทที่ 5) และผลการทดลองจริง (บทที่ 7) สอดคล้องกันอย่างชัดเจน ทำให้มั่นใจในข้อสรุปนี้

8.3 ข้อจำกัด

- Algorithm B ซับซ้อนกว่าในการพัฒนาและทดสอบ เพราะต้องออกแบบ Inverse Operation ให้ถูกต้องครบทุกกรณี ความเสี่ยงต่อบั๊กจึงสูงกว่า Algorithm A ซึ่งพิสูจน์ความถูกต้องได้ง่ายกว่ามาก
- หากการแก้ไขแต่ละครั้งมีขนาดใหญ่ใกล้เคียงกับทั้งเอกสาร ($k \approx n$) ข้อได้เปรียบด้านประสิทธิภาพของ Algorithm B จะลดลงจนใกล้เคียงกับ Algorithm A
- ทั้งสองอัลกอริทึมยังไม่จำกัดขนาดของ Stack จึงอาจใช้หน่วยความจำสะสมมากในระยะยาวหากไม่มีการจำกัดจำนวนประวัติ (History Limit)

8.4 แนวทางพัฒนาต่อ

- จำกัดขนาดของ undoStack ด้วยนโยบาย LRU (ลบ Action ที่เก่าที่สุดเมื่อ Stack เต็ม)
- เพิ่มการบีบอัด/รวม Action ที่เกิดขึ้นติดกันอย่างรวดเร็ว (เช่น การพิมพ์ต่อเนื่องหลายตัวอักษร) ให้เป็น Action เดียวเพื่อลดจำนวนรายการใน Stack
- ขยายรองรับ Rich Text และ Multi-cursor รวมถึงการทำงานร่วมกันหลายผู้ใช้ (Collaborative Editing) ในอนาคต

8.5 สิ่งที่ได้เรียนรู้

- การเลือกโครงสร้างข้อมูลที่เหมาะสม (Stack) และการออกแบบข้อมูลที่จัดเก็บ (สำเนาทั้งฉบับ vs. เฉพาะส่วนต่าง) ส่งผลอย่างมากต่อประสิทธิภาพของระบบจริง แม้ Big-O ของโครงสร้างพื้นฐาน (push/pop) จะเหมือนกันทุกประการ
- การพิสูจน์ความถูกต้องด้วย Invariant ช่วยให้เรามั่นใจได้ว่าอัลกอริทึมทำงานถูกต้องในทุกกรณี ไม่ใช่เฉพาะกรณีที่ทดสอบ
- การวัดผลจริงมีความสำคัญไม่แพ้การวิเคราะห์ทางทฤษฎี เพราะช่วยยืนยันว่าการวิเคราะห์ Big-O สอดคล้องกับพฤติกรรมจริงของโปรแกรม และเผยให้เห็นค่าคงที่ (constant factor) ที่ Big-O ไม่ได้บอกไว้

ภาคผนวก ก: บันทึกการใช้ Generative AI

กลุ่มได้ใช้ Generative AI (Claude, Anthropic) ช่วยในกระบวนการเรียนรู้และร่างเอกสาร/ซอร์สโค้ดเบื้องต้น ตามตารางด้านล่าง สมาชิกทุกคนได้ตรวจสอบ ทดสอบ และทำความเข้าใจเนื้อหาทั้งหมดก่อนนำเสนอ นักศึกษาควรปรับปรุงตารางนี้ให้ตรงกับ Prompt และการทำงานจริงของกลุ่มตนเองก่อนส่งงาน

งานที่ใช้ AI ช่วย	ผลลัพธ์จาก AI (โดยสรุป)	วิธีตรวจสอบ	สิ่งที่แก้ไข/เพิ่มเติม
ร่างโครงสร้างอัลกอริทึม Snapshot และ Command	เสนอ Pseudocode และแนวคิด Inverse Operation เบื้องต้น	จำลองการทำงานที่ละขั้นด้วยตัวอย่างจริง (Trace by hand และรันสคริปต์ Python จำลองตรรกะเดียวกัน)	ปรับปรุงการตรวจสอบขอบเขต (boundary check) และเงื่อนไข การล้าง Redo Stack ให้ครบถ้วน
สร้างซอร์สโค้ด Java เบื้องต้น (Main, Action, SnapshotEditor, CommandEditor)	โค้ดโครงสร้างคลาสตามที่ออกแบบไว้	ตรวจสอบตรรกะด้วยการรันโปรแกรม จำลอง Python ที่มีตรรกะเดียวกันทุก ขั้นตอน และตรวจสอบด้วยสายตาเทียบกับ Pseudocode	เพิ่มการนับ Operation (push/pop/comparisons) และ Auxiliary Space ให้ครบตาม ข้อกำหนด 3.7-3.8
วิเคราะห์ Big-O ของแต่ละฟังก์ชัน	AI เสนอ Time/Space Complexity เบื้องต้น	ตรวจสอบย้อนกลับกับโค้ดจริงที่ละบรรทัด ว่ามีกรวนลูบหรือคัดลอกข้อมูลขนาด เท่าใด	ระบุแยกรณี Best/Average/Worst ให้ชัดเจนขึ้น และเพิ่มคำอธิบายว่าเหตุใด push()/pop() เพียงอย่างเดียวไม่ เพียงพอ
ออกแบบชุดการทดลอง ประสิทธิภาพ	AI เสนอโครงสร้างการทดลองและสร้าง ข้อมูลตัวอย่าง	ตรวจสอบว่าค่าที่ได้สอดคล้องกับทิศทาง ของ Big-O ที่วิเคราะห์ไว้ (A ควรช้ากว่า B เมื่อเอกสารใหญ่ขึ้น)	เพิ่มหมายเหตุด้านระเบียบวิธี เกี่ยวกับข้อจำกัดของสภาพแวดล้อม ที่ไม่มี javac และให้คำแนะนำการ รันซ้ำด้วย Java จริง

การวิเคราะห์ Big-O ของนักศึกษาเอง

กลุ่มได้ตรวจสอบการวิเคราะห์ Complexity ที่ AI เสนอ โดยเทียบกับโค้ดจริงที่ละเมธอด และยืนยันด้วยผลการทดลองในบทที่ 7 ว่าตัวเลข Time/Space Complexity ที่สรุปในบทที่ 5 ถูกต้องและสอดคล้องกับพฤติกรรมจริงของโปรแกรม

Reflection เกี่ยวกับข้อดีและข้อจำกัดของ AI

ข้อดี: AI ช่วยร่างโครงสร้างเริ่มต้นและเอกสารได้รวดเร็ว ทำให้กลุ่มมีเวลามากขึ้นในการทำทำความเข้าใจแนวคิดเชิงลึกและทดสอบโปรแกรม
ข้อจำกัด: AI ไม่สามารถทดแทนความเข้าใจของนักศึกษาได้ โดยเฉพาะการพิสูจน์ความถูกต้อง (Correctness) และการตีความผลการทดลอง ซึ่งต้องอาศัยการตรวจสอบและวิเคราะห์อย่างละเอียดโดยสมาชิกกลุ่มเอง สมาชิกทุกคนจึงต้องสามารถอธิบาย Source Code และเหตุผลของทุกการออกแบบได้ด้วยตนเอง

ภาคผนวก ข: รายงานการแบ่งงานของสมาชิก

ตารางด้านล่างเป็นแม่แบบสำหรับกลุ่มกรอกการแบ่งงานจริง เพื่อให้สอดคล้องกับข้อกำหนดที่ว่าสมาชิกทุกคนต้องมี Commit ใน Repository และสามารถอธิบาย Source Code ในส่วนที่ตนรับผิดชอบได้

ชื่อ-สกุล	รหัสนักศึกษา	หัวข้อ/งานที่รับผิดชอบ	สัดส่วน (%)
		การวิเคราะห์ปัญหา + บทที่ 1-2 ของรายงาน	
		ออกแบบและเขียนโค้ด Algorithm A (Snapshot) + บทที่ 3.1	
		ออกแบบและเขียนโค้ด Algorithm B (Command/Delta) + บทที่ 3.2, 4	
		Test Cases + การทดลองประสิทธิภาพ + บทที่ 5, 7	

		Main.java, การนำเสนอ, สไลด์ + บทที่ 6, 8	
--	--	--	--

หมายเหตุ: กรุณาปรับจำนวนแถวให้ตรงกับจำนวนสมาชิกจริงของกลุ่ม และให้สมาชิกทุกคนเซ็นชื่อรับรองความถูกต้องของการแบ่งงานก่อนส่ง

